



CSCI 232

232 Algorithms & Data Structures

LECTURE: 8

J. L. Pach

OUTLINE

- ✂ Stack memory — recap
- ✂ Heap memory — `malloc` and `free`
- ✂ Traversal — walking a linked list
- ✂ `detachFirst` — step by step
- ✂ `detachValue` — edge cases

STACK VS HEAP

Recap

STACK MEMORY — WHAT YOU ALREADY KNOW

```
void someFunction(void)
{
    Node a;          // born here
    initNode(&a, 10);
    // ...
}                  // dies here — automatically
```



- ✂ Allocated automatically when a function is called.
- ✂ Freed automatically when the function returns.
- ✂ You cannot control the lifetime — the compiler does.
- ✂ Safe, fast, but limited in scope.

A stack node returned by address is a **dangling pointer**.

The memory is gone — you are pointing at nothing valid.

THE PROBLEM STACK CANNOT SOLVE

```
Node* buildList(void)
{
    Node a;
    initNode(&a, 10);
    return &a;    // WRONG — a dies when function returns
}

int main(void)
{
    Node *headPtr = buildList();
    // headPtr points to dead memory
    // undefined behavior
}
```



We need nodes that **outlive** the function that created them.

We need nodes that **exist as long as we decide**.

That is what the heap is for.

HEAP MEMORY

malloc & free

MALLOC — MEMORY ALLOCATION 1/2

```
#include <stdlib.h>
```

```
Node *nodePtr = malloc(sizeof(Node)); // in C++: (Node*) malloc(sizeof(Node));
```



- ✖ `malloc` asks the operating system for a block of memory.
- ✖ Returns a pointer to the first byte of that block.
- ✖ Returns `NULL` if allocation fails — always check.
- ✖ The memory exists until YOU call `free()`.
- ✖ No automatic cleanup — you are responsible.

MALLOC — MEMORY ALLOCATION 2/2

```
Node *nodePtr = malloc(sizeof(Node));  
if (nodePtr == NULL)  
{  
    fprintf(stderr, "Error: malloc failed.\n");  
    return NULL;  
}
```



Never assume `malloc` succeeded. Always guard.

FREE — RELEASING MEMORY 1/2

```
free(nodePtr);  
nodePtr = NULL; // critical — prevents dangling pointer
```



- ✂ `free` returns the memory block to the operating system.
- ✂ After `free` — the pointer still holds the old address.
- ✂ That address is no longer valid — **dangling pointer**.
- ✂ Always set the pointer to `NULL` after `free`.
- ✂ `free(NULL)` is safe — it does nothing.

FREE — RELEASING MEMORY 2/2

```
// WRONG — double free  
free(nodePtr);  
free(nodePtr); // undefined behavior — program may crash
```



```
// SAFE — nullify after free  
free(nodePtr);  
nodePtr = NULL;  
free(nodePtr); // free(NULL) — safe, does nothing
```



STACK VS HEAP — SIDE BY SIDE 1/2

```
// STACK
```

```
Node a;
```

```
initNode(&a, 10);
```

```
// no free needed — dies automatically
```



```
// HEAP
```

```
Node *nodePtr = malloc(sizeof(Node));
```

```
initNode(nodePtr, 10);
```

```
// must free manually
```

```
free(nodePtr);
```

```
nodePtr = NULL;
```



STACK VS HEAP — SIDE BY SIDE 2/2

	Stack	Heap
Lifetime	function scope	until <code>free()</code>
Allocation	automatic	<code>malloc()</code>
Cleanup	automatic	<code>free()</code> — your job
Dangling pointer risk	no	yes

createNode — Stack Replaced by Heap

```
Node a; // stack — works only inside this scope  
initNode(&a, 10);
```



```
Node* createNode(int value) // heap — lives as long as you need  
{  
    Node *newNodePtr = malloc(sizeof(Node));  
    if (newNodePtr == NULL)  
    {  
        fprintf(stderr, "Error: malloc failed.\n");  
        return NULL;  
    }  
    initNode(newNodePtr, value);  
    return newNodePtr; // safe — heap memory outlives function  
}
```



`createNode` returns a pointer to **heap memory**. The caller is responsible for calling `free` eventually.

destroyNode — free + nullify

```
void destroyNode(Node **nodePtrPtr)
{
    if (nodePtrPtr == NULL || *nodePtrPtr == NULL)
        return;
    free(*nodePtrPtr);    // release the memory
    *nodePtrPtr = NULL;   // prevent dangling pointer
}
```



Why `Node **nodePtrPtr` and not `Node *nodePtr` ?

```
// if we only had Node *nodePtr:
void destroyNode_BROKEN(Node *nodePtr)
{
    free(nodePtr);
    nodePtr = NULL; // changes LOCAL copy only
}                  // caller's pointer still dangles

// with Node **nodePtrPtr:
destroyNode(&nodePtr); // caller's pointer becomes NULL ✓
```



TRAVERSAL

Walking a Linked List

The Core Pattern

```
Node *currentPtr = headPtr;

while (currentPtr != NULL)
{
    // do something with currentPtr->value
    currentPtr = currentPtr->nextPtr;
}
```



- ✂ Start at `headPtr` — never modify it directly during traversal.
- ✂ Use a **local cursor** `currentPtr` — a copy of the pointer.
- ✂ At each step: process the node, then move to `nextPtr`.
- ✂ Stop when `currentPtr` reaches `NULL` — end of list.

`headPtr` after traversal is **unchanged**.

`currentPtr` was a local copy — it moved, not `headPtr`.

TRAVERSAL — MEMORY PICTURE

headPtr



[10 | •] —► [20 | •] —► [30 | NULL]

Step 1: currentPtr = headPtr → points to 10

Step 2: currentPtr = currentPtr->nextPtr → points to 20

Step 3: currentPtr = currentPtr->nextPtr → points to 30

Step 4: currentPtr = currentPtr->nextPtr → NULL → stop

PRINTLIST — TRAVERSAL IN PRACTICE

```
int printList(Node *headPtr)
{
    if (headPtr == NULL)
    {
        printf("List is empty.\n");
        return -1;
    }

    Node *currentPtr = headPtr;

    while (currentPtr != NULL)
    {
        printf("%d\n", currentPtr->value);
        currentPtr = currentPtr->nextPtr;
    }
    return 0;
}
```



listLength — Counting with Traversal

```
int listLength(Node *headPtr)
{
    int count = 0;
    Node *currentPtr = headPtr;

    while (currentPtr != NULL)
    {
        count++;
        currentPtr = currentPtr->nextPtr;
    }
    return count;
}
```



Same pattern — different action at each node.

Traversal is always the same loop.

Only what you do **inside** the loop changes.

_findLast — Walk to the End

```
static Node* _findLast(Node *headPtr)
{
    if (headPtr == NULL)
        return NULL;

    Node *currentPtr = headPtr;

    while (currentPtr->nextPtr != NULL)
        currentPtr = currentPtr->nextPtr;

    return currentPtr;
}
```



Note the condition — `currentPtr->nextPtr != NULL` .

We stop **at** the last node, not **after** it.

We need to return a pointer to it — so we must stop before going past it.

STEP BY STEP

detachFirst

DETACHFIRST — WHAT IT DOES

BEFORE:

headPtr —► [10 | •] —► [20 | •] —► [30 | NULL]

AFTER detachFirst(&headPtr):

headPtr —► [20 | •] —► [30 | NULL]

returned —► [10 | NULL] ← nextPtr cleared

The node is **removed from the list** but NOT freed.

The caller decides what to do with it:

✂ Stack node → do nothing

✂ Heap node → call `destroyNode`

DETACHFIRST — IMPLEMENTATION

```
Node* detachFirst(Node **headPtrPtr)
{
    if (headPtrPtr == NULL || *headPtrPtr == NULL)
        return NULL;

    Node *detachedPtr = *headPtrPtr;    // save the node
    *headPtrPtr      = (*headPtrPtr)->nextPtr; // advance head
    detachedPtr->nextPtr = NULL;         // clear the link

    return detachedPtr;
}
```



Three steps — always in this order:

1. Save pointer to the node we are removing.
2. Move headPtr forward to the next node.
3. Clear nextPtr of the detached node — it is now isolated.

DETACHFIRST — STEP BY STEP IN MEMORY

Step 1 — save:

detachedPtr —► [10 | ●] —► [20 | ...]

headPtr —► [10 | ●] —► [20 | ...]

Step 2 — advance head:

detachedPtr —► [10 | ●] —► [20 | ...]

headPtr —► [20 | ...]

Step 3 — clear link:

detachedPtr —► [10 | NULL]

headPtr —► [20 | ●] —► [30 | NULL]

After step 3 — the detached node is completely isolated.

The list starts at 20. The caller has 10.

DELETEFIRST — DETACH + DESTROY

```
int deleteFirst(Node **headPtrPtr)
{
    Node *detachedPtr = detachFirst(headPtrPtr);
    if (detachedPtr == NULL)
    {
        fprintf(stderr, "Error: list is empty.\n");
        return -1;
    }
    destroyNode(&detachedPtr);
    return 0;
}
```



`deleteFirst` is just two lines of logic:

detach → the list no longer owns the node

destroy → we no longer own the node either

EDGE CASES

detachValue

detachValue — Three Cases

List: [10] —► [20] —► [30] —► NULL

Case 1 — value is at the HEAD:

`detachValue(&headPtr, 10)`

→ call `detachFirst` — already written ✓

Case 2 — value is in the MIDDLE or at the END:

`detachValue(&headPtr, 20)`

`detachValue(&headPtr, 30)`

→ traverse to find the node BEFORE it

→ bypass it

Case 3 — value NOT FOUND:

`detachValue(&headPtr, 99)`

→ return NULL

Case 1 is special because there is no previous node.
Always handle it first — then traverse for cases 2 and 3.

detachValue — Middle/Last Bypass

BEFORE detachValue(&headPtr, 20):

headPtr —► [10 | •] —► [20 | •] —► [30 | NULL]



we want this one

currentPtr stops HERE:

headPtr —► [10 | •] —► [20 | •] —► [30 | NULL]



currentPtr

currentPtr->nextPtr == node with value 20

AFTER bypass:

headPtr —► [10 | •] —————► [30 | NULL]

returned —► [20 | NULL]

DETACHVALUE — IMPLEMENTATION

```
// haha
```



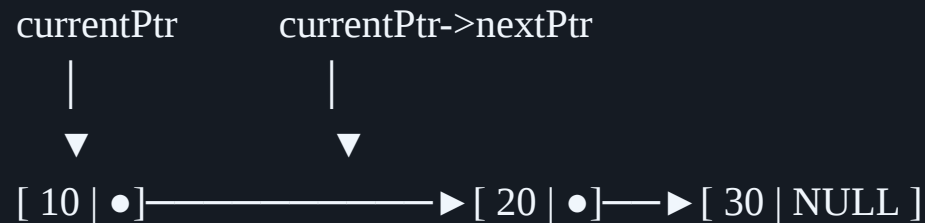
detachValue — Why `currentPtr->nextPtr`

```
while (currentPtr->nextPtr != NULL)
{
    if (currentPtr->nextPtr->value == value)
```



We look **one node ahead** — not at `currentPtr` itself.

Why? Because to remove a node we need to modify the **previous** node's `nextPtr`. If we stop at the node we want to remove — it is too late, we have lost access to its predecessor.



We check value at `currentPtr->nextPtr`.

If it matches — we can do:

```
currentPtr->nextPtr = currentPtr->nextPtr->nextPtr
```

SUMMARY

Function	Memory	What it does
createNode	heap	malloc + initNode
destroyNode	heap	free + NULL
detachFirst	stack + heap	removes head, no free
detachValue	stack + heap	finds and removes, no free
deleteFirst	heap	detachFirst + destroyNode
deleteValue	heap	detachValue + destroyNode

detach = remove from list — caller decides memory

delete = remove from list AND free — heap only

ONE RULE TO REMEMBER

detach → structure operation → stack and heap

delete → memory operation → heap only

delete = detach + destroyNode

If you understand `detach` — `delete` is just two lines.

If you understand `detachFirst` — `detachValue` is the same loop with one extra case at the head.

THANK

You